



MDMC

Master in Data Management
and Curation



Scientific Programming Environment – Introduction to UNIX-like Operating Systems

Trieste, September 23rd 2025



- The beginnings (1970-1980)
 - 1970: Start of UNIX development at Bell Labs by Ken Thompson, Dennis Ritchie, Brian Kernighan et al.
 - 1978: First release of the Berkeley Software Distribution (BSD).
 - 1979: First commercial distributions of UNIX by Santa Cruz Operations (SCO).

- The turmoil years (1980-1990)
 - 1982: TCP/IP development and refinement starts on BSD 4.x.
 - 1982: Breakup of American Telephone & Telegraph Company ("Ma Bell") monopoly into multiple local companies. AT&T can sell UNIX and starts a series of lawsuits and litigations.
 - 1983: Richard Stallman founds the GNU Project.
 - 1987: Andrew Tanenbaum, a professor at Vrije Universiteit in Amsterdam, publishes MINIX source code in conjunction with his book "Operating Systems: Design and Implementation".
 - 1988: The first POSIX standard is published by IEEE.

- The rise of Linux (1991-2020)
- 1991: Linus Torvalds, a student of the University of Helsinki, starts working on the Linux kernel.
- 1992: Linux is relicensed under the GNU GPL.
- 1994: Linux 1.0 is released.
- 1996: Linux 2.0 is released with support for more than one processor (SMP).
- Lots of stuff...
- 2017: All of the TOP500 supercomputers run Linux.

- The UNIX principles
- Write programs that do one thing and do it well.
- Write programs to work together.
- Write programs to handle text streams.
- Choose portability over efficiency.
- Store data in plain text files.
- Everything is a file.

- The UNIX principles
- Write programs that do one thing and do it well.
- E.G.:
 - cut: cut lines at specified points.
 - split: split a text file at specified lines.

- The UNIX principles
- Write programs to work together.
- Write programs to handle text streams.
- E.G.: get the third field of a.csv and the fifth field of b.csv
- ```
$ paste -d, <(cut -d, -f3 a.csv) <(cut -d, -f5 b.csv)
```

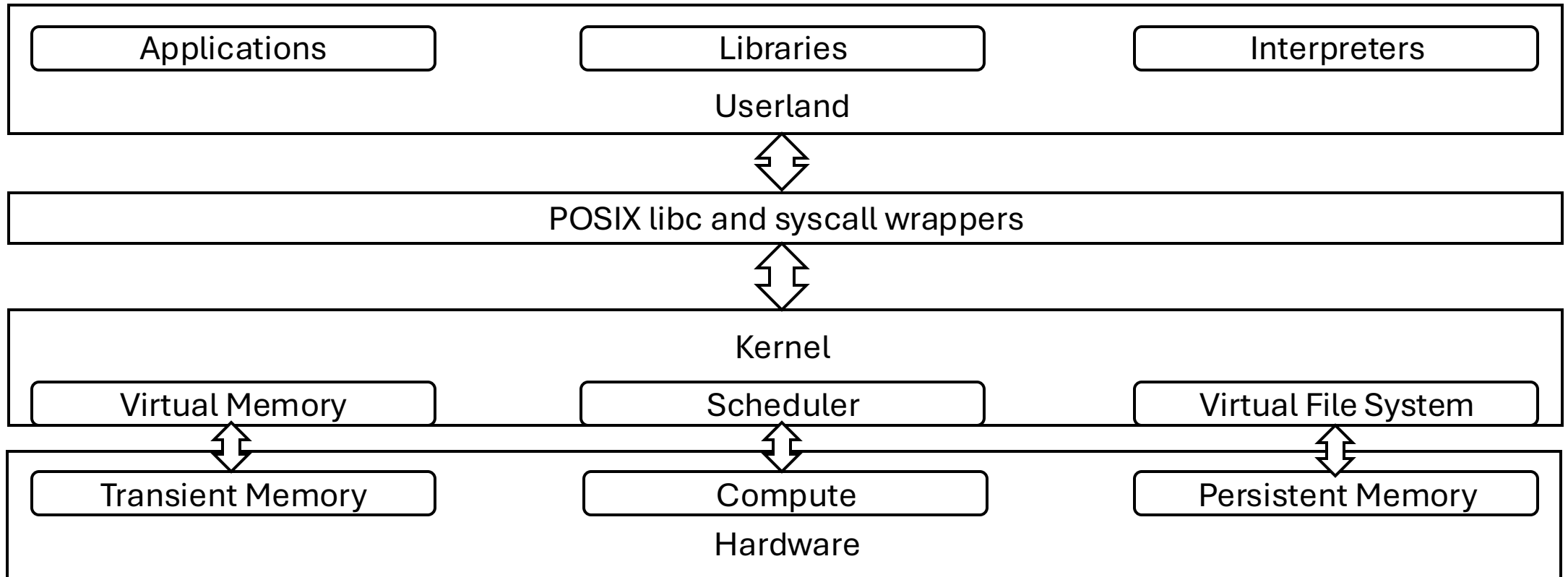
- The UNIX principles
- Choose portability over efficiency.
- Reason: CPU time is cheap, human time is not. "Portability over efficiency" means less headaches when your underlying system changes.



- The UNIX principles
- Store data in plain text files.
- Reason: plain text files ensure that you can read your data even if you lose access to the software that created it.

- The UNIX principles
- Everything is a file.
- Reason: give applications a consistent interface capable of operating on data regardless if it's from a file, from the hardware or from another computer.

- Overview of a UNIX-like OS



- The Virtual Filesystem (VFS)
- The word Filesystem in UNIX can mean a lot of different things:
  - The logical structure holding data in files and directories.
  - A particular way of storing data on physical media.
  - An abstraction for accessing certain kernel structures at the application level.
  - A network protocol to distribute data and metadata to multiple computers.

- The Virtual Filesystem (VFS)
- The Virtual Filesystem (VFS) is a subsystem of the kernel that abstracts the complex underlying of OS-level data management to applications and presents a cohesive interface (via libc) to read, write and query data and metadata.
- This interface is defined by the POSIX standard and you can read about its implementation in the Linux kernel in Chapter 4 of "The Linux Programming Interface" (Kerrisk 2010 - No Starch Press - ISBN 9781593272203)

[https://nostarch.com/download/samples/LinuxProgrammingInterface\\_Chapter4\\_Updated.pdf](https://nostarch.com/download/samples/LinuxProgrammingInterface_Chapter4_Updated.pdf)

- Files in UNIX
- "Everything is a file" doesn't mean that everything is stored as a meaningless sequence of ones and zeros without any kind of order.
- Files have some properties that are independent from the data they contain like:
  - A name
  - An owner
  - Permissions
  - Etcetera

- Files in UNIX
- Other than regular files there are other kinds of files:
  - Directories
  - Links
  - Devices
  - Named pipes
  - UNIX sockets

- Files in UNIX - Directories
- Directories are special files that contain pointers to other files.
- The ls command reads the contents of a directory and prints them out.
- Directories store the names of the files they contain and the relative pointer to the associated inode (a data structure containing owner, group, permissions, type and data block addressing information about the file itself).



- Files in UNIX - Links
- Since information regarding the name of a file is stored separately from the rest of the metadata, it is possible to have multiple entries of a directory pointing to the same file with different names and/or directories in different positions of the filesystem that can contain the same file.
- These entries are called Hard Links.
- Hard Links are limited to the same filesystem (usually tied to a specific physical media).

- Files in UNIX - Links
- To overcome hard links limitations one can use a Symbolic Link (symlink).
- Symlinks are special files that contain the path to the file they point at. They have no notion about inodes and the associated metadata.
- Symlinks can be unresolvable by the OS at a specific moment in time (E.G. because they point at a file in a network share and there's no connection). These are called broken links.

- Ownership and permissions
- UNIX is designed to have multiple users, each with their own files.
- Users can also be members of one or more groups.
- Each file in UNIX has an owner (user) and it also belongs to a group.
- Each file also has a set of permissions for the owner, the group and the rest of the world (others).
- A file can be read (r), written (w) or executed (x).

- Ownership and permissions
- Under normal conditions only the owner of a file can change the permissions of said file. However there's a user that's special: the root user.
- The normal checks regarding permissions and ownership don't apply to root. Therefore root can do everything to every file in the system.
- The root user is also called the Superuser due to those powers.

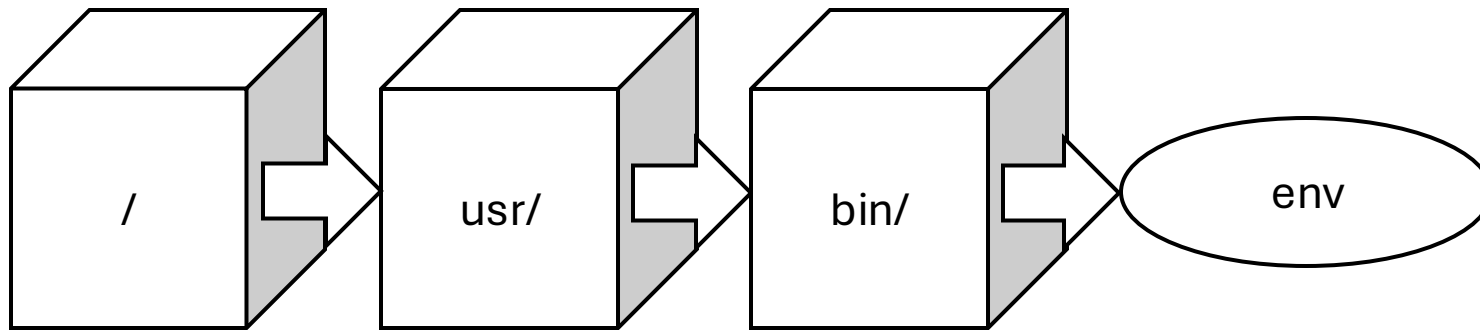
- Ownership and permissions

| Owner (u)          | Group (g)          | Others (o)         |
|--------------------|--------------------|--------------------|
| Read Write eXecute | Read Write eXecute | Read Write eXecute |

- Readable and writable by owner, read only for anybody else: rw-r--r--
- Readable and writable by owner and group, read only for anybody else: rw-rw-r--
- Standard Executable permissions (need to be readable): r-xr-xr-x

- The Filesystem Hierarchy
- Each UNIX systems has a special directory (usually owned by root) that acts as the starting point for the whole file hierarchy.
- This is the root directory and it's denoted by a single / symbol.
- Each file in a UNIX system can be reached starting by /, appending the name of the directory (or directories) that lead to the file separated by / and finally the name of the file itself.

- The Filesystem Hierarchy
- E.G.: /usr/bin/env



- The Home Directory
- Each user has a special directory associated with their account: the home directory.
- The home directory is the first directory where a user lands after logging in to a UNIX system. Usually the user is also the owner of their home directory.
- Applications usually save the user's preferences and settings inside the home directory.



- The Shell

- The shell is the outer layer of the operating system; the interface between the user and the rest of the system.
- There are two main types of shell: textual and graphical.
- UNIX pre-dates graphical shells so all of the early development and design is text-oriented.
- To this day the textual shells are routinely used by UNIX administrators and users because textual shells allow to compose commands and automate certain tasks.

Thanks for your attention!

Any questions?